Python for Data Analysis: Practice Assignment

Dataset: Student Life & Daily Habits (student_life_dataset_200.csv)

Welcome to your Python practice worksheet! This assignment uses 'student_life_dataset_200.csv', which contains data for 200 college students tracking their major, study hours, sleep habits, caffeine intake, GPA, job status, and daily screen time. Follow the instructions below to complete each exercise using Pandas and NumPy.

Part 1: Basic Dataset Inspection (Pandas)

Q1. Load the dataset 'student_life_dataset_200.csv' into a Pandas DataFrame named 'df' and display its first 5 rows.

```
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('student_life_dataset_200.csv')

# Show the first 5 rows
print(df.head())
```

Q2. Print out a summary of the columns, data types, and non-null counts to check if there is any missing data.

```
# Dataset structural summary
print(df.info())
```

Q3. Generate the overall descriptive statistics (mean, min, max, standard deviation) for all numerical metrics in the dataset.

```
# Descriptive statistics
print(df.describe())
```

Q4. Find out how many unique college majors are represented in this dataset and count how many students belong to each major.

```
# Count students per major
print(df['Major'].value_counts())
```

Part 2: Conditional Filtering & Grouping (Pandas)

Q5. Filter the DataFrame to view only the students who study more than 30 hours per week.

```
# Hard-working students filter
heavy_studiers = df[df['Study_Hours_Per_Week'] > 30]
print(heavy_studiers.head())
```

Q6. Find the students who are highly caffeinated and high-achieving: filter for those drinking more than 15 cups of coffee per week AND keeping a GPA higher than 3.5.

```
# Filtering with multiple conditions
caffeinated_achievers = df[(df['Coffee_Cups_Per_Week'] > 15) & (df['GPA'] > 3.5)]
```

```
print(caffeinated_achievers)
```

Q7. Calculate and compare the average GPA and sleep hours between students who hold a part-time job and those who do not.

```
# Grouping by employment status
job_comparison = df.groupby('Part_Time_Job')[['GPA', 'Sleep_Hours_Per_Night']].mean()
print(job_comparison)
```

Q8. Add a new column called 'Total_Weekly_Screen_Time' which converts daily screen time into weekly screen time (multiply daily screen time by 7).

```
# Creating a new feature column
df['Total_Weekly_Screen_Time'] = df['Screen_Time_Hours_Per_Day'] * 7
print(df[['Screen_Time_Hours_Per_Day', 'Total_Weekly_Screen_Time']].head())
```

Part 3: Mathematical Operations (NumPy)

Q9. Convert the 'GPA' column into a standalone NumPy array, and use NumPy to find the overall median GPA.

```
# Convert column to NumPy array
gpa_array = df['GPA'].to_numpy()

# Calculate median
median_gpa = np.median(gpa_array)
print(f"Median GPA of all students: {median_gpa}")
```

Q10. Extract 'Study_Hours_Per_Week' as a NumPy array and determine the 90th percentile cutoff value for study hours.

```
# Calculate percentile using NumPy
study_array = df['Study_Hours_Per_Week'].to_numpy()
p90_study = np.percentile(study_array, 90)
print(f"90th Percentile cutoff for study hours: {p90_study}")
```